

PROJECT PROPOSAL

By

Pankaj Kumawat

**Comparative Analysis of Custom CNN and Transfer Learning Models
with Explainable AI for Fresh and Stale Food Classification**

M.Sc. Data Science

University of Europe for Applied Sciences, Potsdam

Supervisor: Prof. Raja Hashim Ali

Date: 07.06.2026

1. Project Title

Comparative Analysis of Custom CNN and Transfer Learning Models with Explainable AI for Fresh and Stale Food Classification

2. Background and Motivation

Food quality is one of those problems that sits at the intersection of public health, economics, and everyday life. Spoiled produce causes real harm both financially and in terms of safety yet the dominant approach to catching it has long been manual inspection. Someone physically looks at a fruit or vegetable and decides whether it is still good. That works at small scale, but it breaks down quickly in modern supply chains where thousands of items move through warehouses every hour. Human inspectors get tired, make inconsistent calls, and simply cannot keep pace with the volume.

Deep learning, particularly CNNs, has changed what is possible here. These models have proven themselves across a huge range of visual tasks, and food quality is a natural fit: freshness and spoilage are largely visible in colour, texture, and surface condition. Rather than training a model from scratch every time, transfer learning lets us borrow feature representations that were built up on millions of ImageNet images. Models like MobileNetV2 and ResNet50 start with a substantial head start, which means they can reach strong performance even when domain-specific labelled data is limited.

There is, however, a catch. A model that just outputs 'fresh' or 'rotten' without any explanation is difficult to trust in practice. Food safety decisions often have regulatory and commercial consequences, and stakeholders need to understand why a classification was made, not just what it was. This is where XAI tools like Grad-CAM and SHAP become genuinely useful they allow us to peek inside the model and verify that it is attending to the right visual cues rather than picking up on irrelevant background artefacts.

This project brings these pieces together: a comparison of three model architectures for 22-class fresh/stale food classification, with a full XAI analysis layered on top.

3. Problem Statement

Manually inspecting food quality is slow, inconsistent, and impossible to scale across modern distribution networks. What is needed is a system that can look at an image of a fruit or vegetable and reliably decide whether it is fresh or spoiled and do so across a wide range of produce types, under varying conditions, and at a speed that actually matches real-world throughput.

This project tackles that problem directly, though it comes with a set of concrete challenges that need to be addressed:

- The classification task spans 22 fine-grained categories (11 fresh and 11 stale variants), which is considerably more complex than simple binary fresh/rotten detection

- Some fresh and stale items look genuinely similar, especially in early stages of spoilage, making automated discrimination harder than it might initially appear
- Class sizes vary across the dataset, which can push models to favour majority classes if imbalance is not explicitly handled
- Deep learning models are often treated as black boxes, which creates real problems in food safety contexts where decisions need to be explainable and auditable
- Different architectures come with different computational costs, and understanding those trade-offs matters for realistic deployment decisions

4. Selected Dataset

Dataset Name

Fresh and Stale Classification Dataset

Dataset Source

Available on Kaggle: <https://www.kaggle.com/datasets/swoyam2609/fresh-and-stale-classification>

Reason for Selection

- The dataset is freely available and well-organised, with pre-existing train, validation, and test splits that make experimental setup straightforward
- Images were captured under real-world conditions different backgrounds, lighting, and stages of spoilage rather than in controlled lab settings, which makes the task genuinely challenging and practically relevant
- Twenty-two classes covering eleven different types of produce gives enough variety to test whether models generalise across food types, not just within one
- The volume of images is sufficient to support both from-scratch CNN training and transfer learning without severe data constraints
- The domain directly maps to practical food safety and supply chain use cases, making the findings relevant beyond just an academic exercise
- The visual nature of freshness cues discolouration, mould, texture changes makes this dataset well-suited to XAI analysis using Grad-CAM and SHAP

5. Dataset Description

Attribute	Description
Dataset Type	Image Dataset
Task	Multi-class Classification (22 classes)
Number of Classes	22 (11 fresh + 11 stale categories)
Classes	Fresh Apples, Banana, Bittergourd, Capsicum, Cucumber, Okra, Oranges, Potato, Tomato + Rotten variants of each

Image Format	JPG / JPEG / PNG / BMP / WEBP
Input Size	224 × 224 pixels after preprocessing
Image Type	RGB colour photographs of fruits and vegetables
Dataset Split	Train: ~70% Validation: ~15% Test: ~15%

Number of Classes

The dataset is split into two broad groupings, fresh and stale, across eleven types of produce, giving 22 classes in total:

- Fresh: freshapples, freshbanana, freshbittergourd, freshcapsicum, freshcucumber, freshokra, freshoranges, freshpatato (potato), freshpotato, freshtamto (tomato), freshtomato
- Stale / Rotten: rottenapples, rottenbanana, rottenbittergourd, rottencapsicum, rottencucumber, rottenokra, rottenoranges, rottenpatato, rottenpotato, rottentamto, rottentomato

One thing worth noting: the dataset contains two naming variants for certain items freshpatato / freshpotato and freshtamto / freshtomato which appear to represent different subsets or collection batches. These are treated as separate classes throughout the project.

Image Type

All images are RGB colour photographs taken in real-world environments, which means there is genuine variation in backgrounds, lighting angles, and how advanced the spoilage is. During preprocessing, every image is resized to 224 x 224 pixels to meet the input requirements of all three model architectures.

Class Distribution

The distribution of images across training, validation, and test splits varies noticeably between classes. Some produce categories have substantially more examples than others, which is a pattern that needs to be managed carefully to avoid biased predictions.

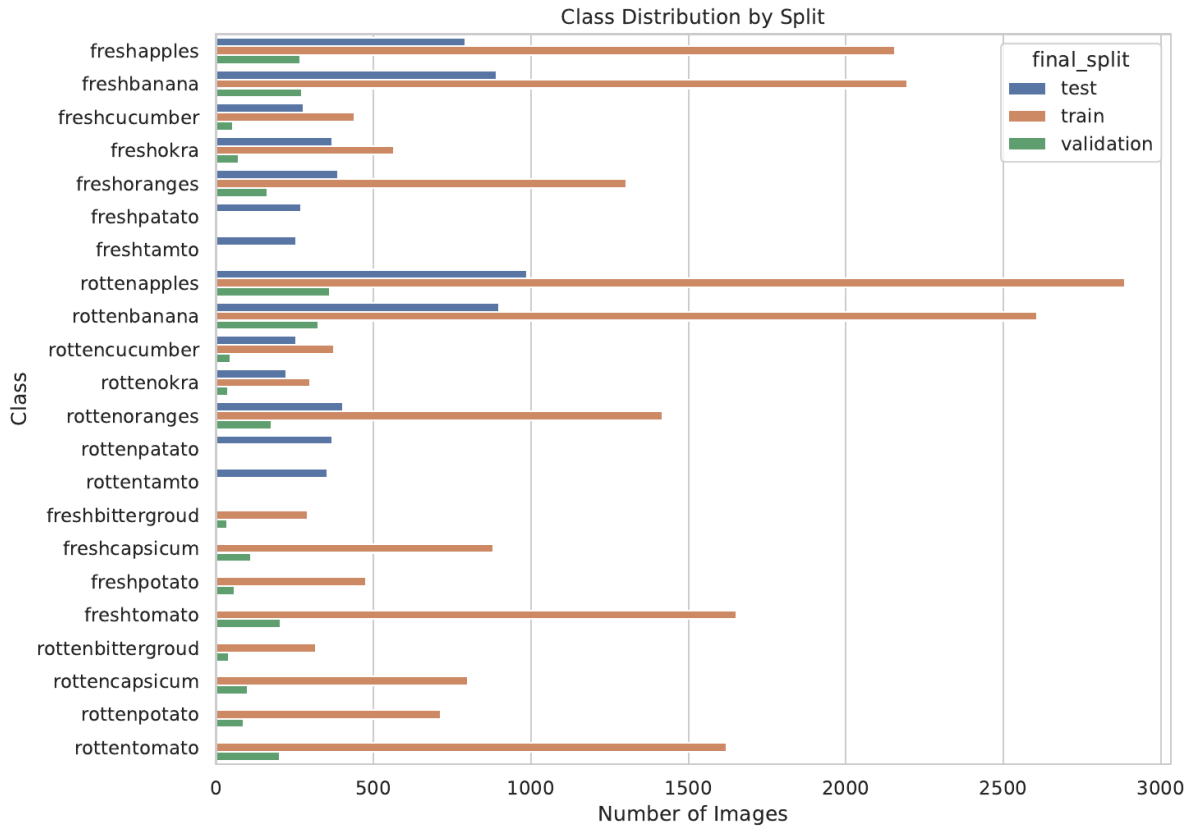


Figure: Class Distribution Across Dataset Splits

Sample Dataset Images

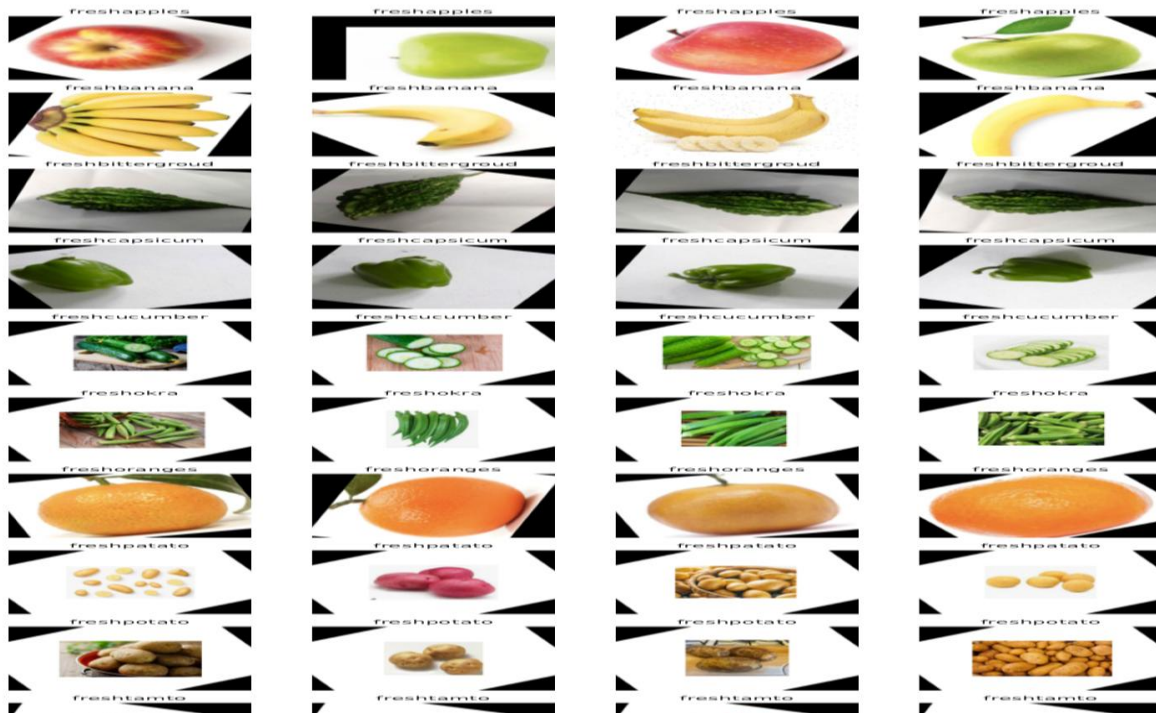


Figure 1: Dataset Samples — four images per class across all 22 categories

6. Research Questions

RQ1: How well can CNN-based models actually classify fruits and vegetables as fresh or stale across all 22 categories when trained on this publicly available dataset?

RQ2: Among the three architectures custom CNN, MobileNetV2, and ResNet50, which one strikes the best balance between classification accuracy and how long it takes to train?

RQ3: To what extent do decisions around preprocessing, augmentation, and class-imbalance handling affect model robustness and per-class performance?

RQ4: When we look at the Grad-CAM heatmaps, are the models actually focusing on the parts of the image that matter discolouration, mould, texture changes or are they picking up on irrelevant background features?

RQ5: Where do these models break down, and what does that mean for anyone thinking about deploying a system like this in a real food inspection setting?

7. Expected Results

Drawing on what is known about transfer learning in image classification, a few outcomes seem fairly likely going into this project:

- MobileNetV2 and ResNet50 should comfortably outperform the custom CNN on both accuracy and weighted F1-score pre-trained ImageNet features give them a head start that is hard to match from scratch
- ResNet50 is the most likely candidate for the top overall accuracy, given its depth and the strength of its residual feature extraction
- MobileNetV2 should be the most efficient in terms of training time relative to accuracy it was designed for exactly this kind of trade-off
- The custom CNN provides a useful baseline, even if it finishes last on accuracy metrics; it helps quantify what transfer learning is actually adding
- Grad-CAM analysis should show that the better-performing models are attending to the food item itself looking at surface texture and colour for freshness, and discolouration or mould patterns for staleness
- SHAP pixel attributions are expected to align reasonably well with human intuition about what makes a fruit look rotten
- The hardest cases will likely be visually similar fresh and stale pairs, and potentially some confusion between the naming variants like freshtamto and freshtomato

8. Proposed Methodology

8.1 Data Collection

The Fresh and Stale Classification dataset is downloaded from Kaggle. It comes with a pre-organised directory structure separating train, validation, and test images, which simplifies the

setup considerably. A recursive directory scan is used to collect all image file paths, and class labels are inferred automatically from the folder names stripping out known split directory names to get to the actual class.

8.2 Data Preprocessing

Before any image reaches a model, it goes through the following steps:

- Image discovery: the dataset directory is scanned recursively for all supported file formats (.jpg, .jpeg, .png, .bmp, .webp)
- Label inference: the class name is pulled from the parent folder, after filtering out any known split-level directory names like 'train' or 'test'
- Split assignment: where the dataset already has explicit train/validation/test folders, those are used directly; if not, stratified random splitting handles it
- Image resizing: every image is resized to 224 x 224 pixels using TensorFlow's resize operation
- Pixel normalisation: a Rescaling layer inside the model brings pixel values from their original 0–255 range down to [0, 1]
- Class weights: to deal with the imbalance across categories, inverse-frequency weights are computed from the training distribution and passed to the model during training

8.3 Dataset Splitting

Where the dataset already has explicit directory splits, those are used as-is. In cases where no splits exist, stratified random splitting is applied:

- Training Set: approximately 70%
- Validation Set: approximately 15%
- Test Set: approximately 15%

8.4 Data Augmentation

Augmentation runs on-the-fly during training using standard Keras preprocessing layers, so validation and test images are never touched by it. The pipeline is deliberately kept modest — aggressive augmentation can hurt as much as it helps when classes already have visual variation:

- Random horizontal flip (50% probability)
- Random rotation (up to $\pm 10\%$)
- Random zoom (up to $\pm 12\%$)
- Random contrast adjustment (up to $\pm 15\%$)

8.5 Model Training

All three models share the same training setup: Adam optimiser, Sparse Categorical Cross-Entropy loss. For the two transfer learning models, training runs in two distinct phases. First, the base model's weights are kept frozen while only the new classification head learns. Then, the top portion of the base model is unfrozen and the whole network is fine-tuned with a much lower learning rate. Early stopping and model checkpointing are used throughout the best weights on

the validation set are saved, and training stops automatically if there is no improvement for a set number of epochs.

9. CNN Model Design

The custom CNN is assembled using the Keras Functional API. The architecture builds depth gradually through a series of convolutional blocks, with each block doubling the number of filters:

- Input: 224 x 224 x 3 RGB images
- Data augmentation block (active only during training): RandomFlip, RandomRotation, RandomZoom, RandomContrast
- Rescaling: pixels normalised to [0, 1]
- Four convolutional blocks with filter counts [32, 64, 128, 256]: each block runs two Conv2D layers (3x3 kernels, ReLU activation) with Batch Normalisation, then MaxPooling2D to reduce spatial dimensions
- A final Conv2D layer with 384 filters and 3x3 kernels, named `custom_last_conv` so that Grad-CAM can target it
- Global Average Pooling, collapsing the spatial dimensions into a single feature vector
- Dropout at rate 0.40 to regularise
- A Dense output layer with 22 units and Softmax activation

The model is compiled with Adam at learning rate 1e-3. Class weights are applied during training to counteract class imbalance.

Figure 2: Model Design Workflow



Figure 2: Model Design Workflow — Input → Training → Prediction → Evaluation → Explainability

10. Transfer Learning Models

MobileNetV2

MobileNetV2 was built with efficiency in mind inverted residual blocks and linear bottlenecks let it achieve strong accuracy with far fewer parameters than heavier architectures. That makes it particularly appealing in settings where training time or deployment resources are limited.

Setup for this project:

- Base: MobileNetV2 pre-trained on ImageNet, with weights frozen during the initial training phase
- Input preprocessing: pixel values scaled to $[-1, 1]$ using MobileNetV2's native preprocessing
- Augmentation is applied before preprocessing
- Classification head: GlobalAveragePooling2D $\rightarrow$ Dropout(0.30) $\rightarrow$ Dense(22, softmax)
- Fine-tuning phase: the top layers of the base are unfrozen and trained with a reduced learning rate

The main strengths expected here are training speed and a low parameter count, making it the most deployment-friendly of the three options.

ResNet50

ResNet50 takes a different approach 50 layers deep, with skip connections that allow gradients to flow across layers without degrading. This design makes it possible to train very deep networks without hitting the vanishing gradient problem that plagued earlier architectures.

Setup for this project:

- Base: ResNet50 pre-trained on ImageNet, weights frozen initially
- Input preprocessing: channel-wise mean subtraction as per ResNet50's native preprocessing
- Classification head: GlobalAveragePooling2D $\rightarrow$ Dropout(0.30) $\rightarrow$ Dense(22, softmax)
- Fine-tuning: the upper convolutional blocks are unfrozen in the second phase with a low learning rate

ResNet50 is expected to produce the strongest raw classification performance, particularly on the harder cases where fresh and stale items look similar.

11. Evaluation Metrics

Metric	What it tells us
Accuracy	Overall share of correct predictions across all 22 classes
Precision	Of everything predicted as a given class, how many actually belong there
Recall	Of everything that actually belongs to a class, how many were correctly found
F1-Score (weighted)	Harmonic mean of precision and recall, weighted by class size to account for imbalance

Confusion Matrix	A full breakdown of where each class was correctly predicted and where it was confused with others
ROC / AUC	How well each model separates classes across different decision thresholds
Accuracy / Loss Curves	Epoch-by-epoch view of how training and validation performance evolved
Training Time (s)	Total wall-clock time from start to best checkpoint — a proxy for computational cost
Total Parameters	Model size and complexity indicator
Grad-CAM Heatmaps	Visual map of which image regions drove each prediction
SHAP Values	Pixel-level attribution showing how individual image patches pushed the prediction in each direction

All classification metrics are evaluated on the held-out test set data the models have never seen during training or validation. For multi-class metrics, weighted averages are used to account for the fact that some classes have more examples than others. Training time and parameter counts are logged during the training runs.

12. Expected Figures and Tables

Figure 2: Model Design Workflow

Figure 2: Model Design Workflow



Figure 2: Model Design Workflow

Figure 3: Accuracy and Loss Curves

Training and validation accuracy and loss curves plotted across 25+ epochs for all three models. The transfer learning models are expected to climb faster and plateau higher than the custom CNN, which typically needs more epochs to converge from scratch.

Figure 3: Accuracy and Loss Curves

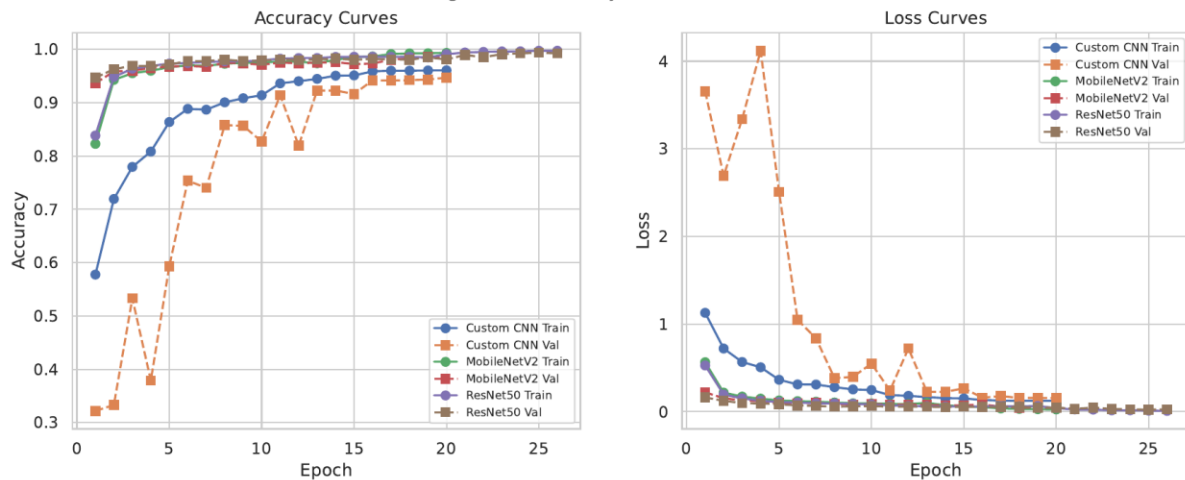


Figure 3: Accuracy and Loss Curves — Custom CNN, MobileNetV2, and ResNet50

Figure 4: Confusion Matrix Comparison

Side-by-side confusion matrices for each model on the test set. These give a clear picture of which classes are being confused with one another, which is often more informative than a single accuracy number.

Figure 4: Confusion Matrix Comparison

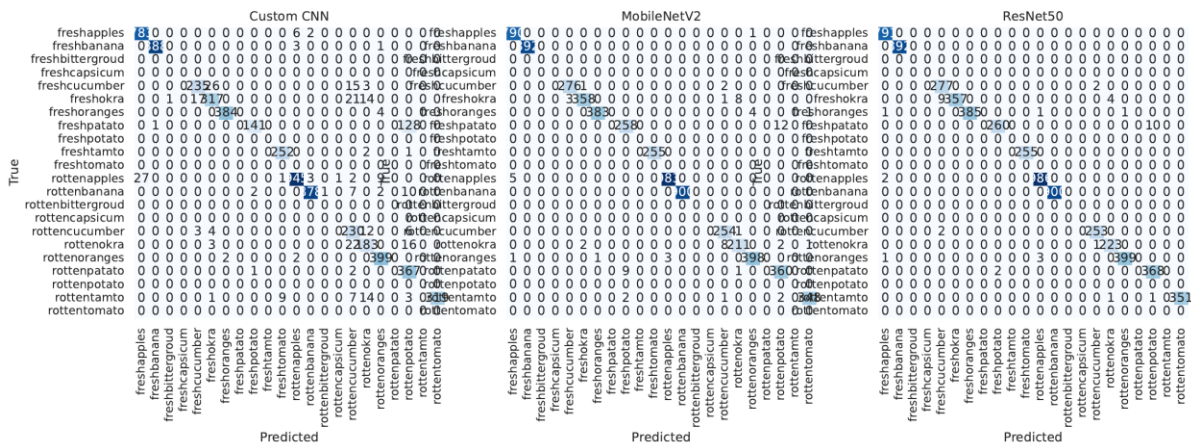


Figure 4: Confusion Matrix Comparison — all three models

Figure 5: Grad-CAM Correct Prediction Examples

Grad-CAM heatmaps overlaid on a correctly classified image, shown for all three models. The coloured regions — warmest where the model paid most attention — should ideally land on the food item rather than the background. Strong models are expected to zero in on the produce itself.

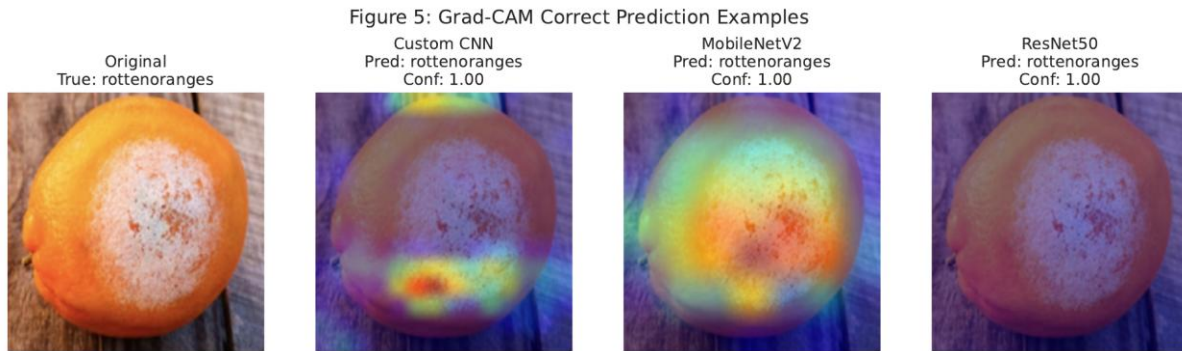


Figure 5: Grad-CAM Correct Prediction Examples

Figure 6: Grad-CAM Misclassification Examples

The same Grad-CAM treatment applied to misclassified examples. These are the interesting cases: seeing where the model looked when it got it wrong often reveals whether the mistake was a reasonable one (ambiguous image) or whether the model was attending to the wrong features entirely.

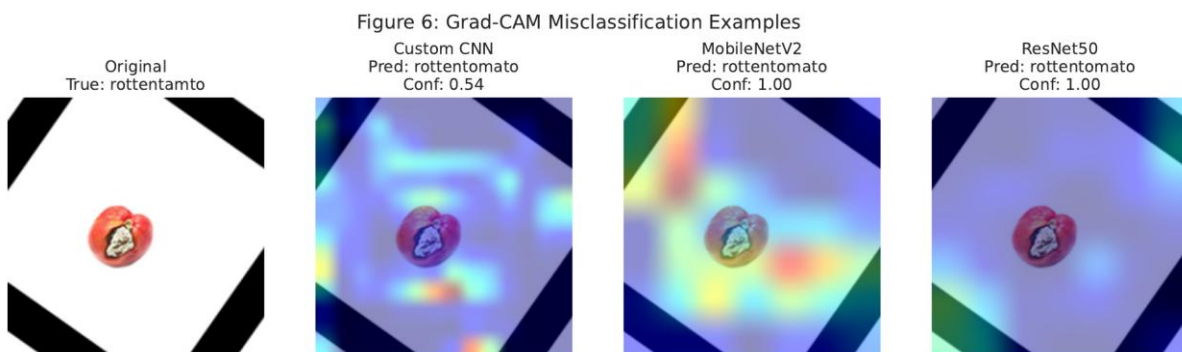
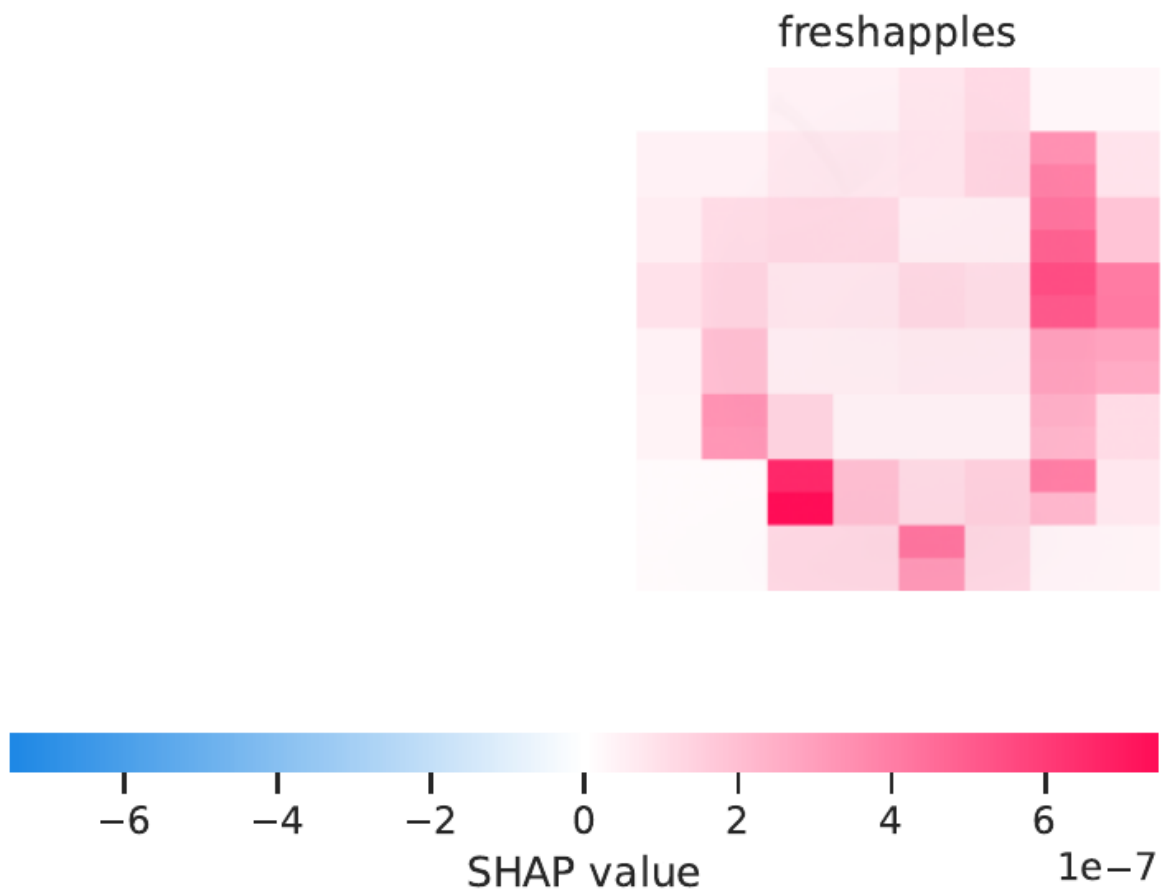
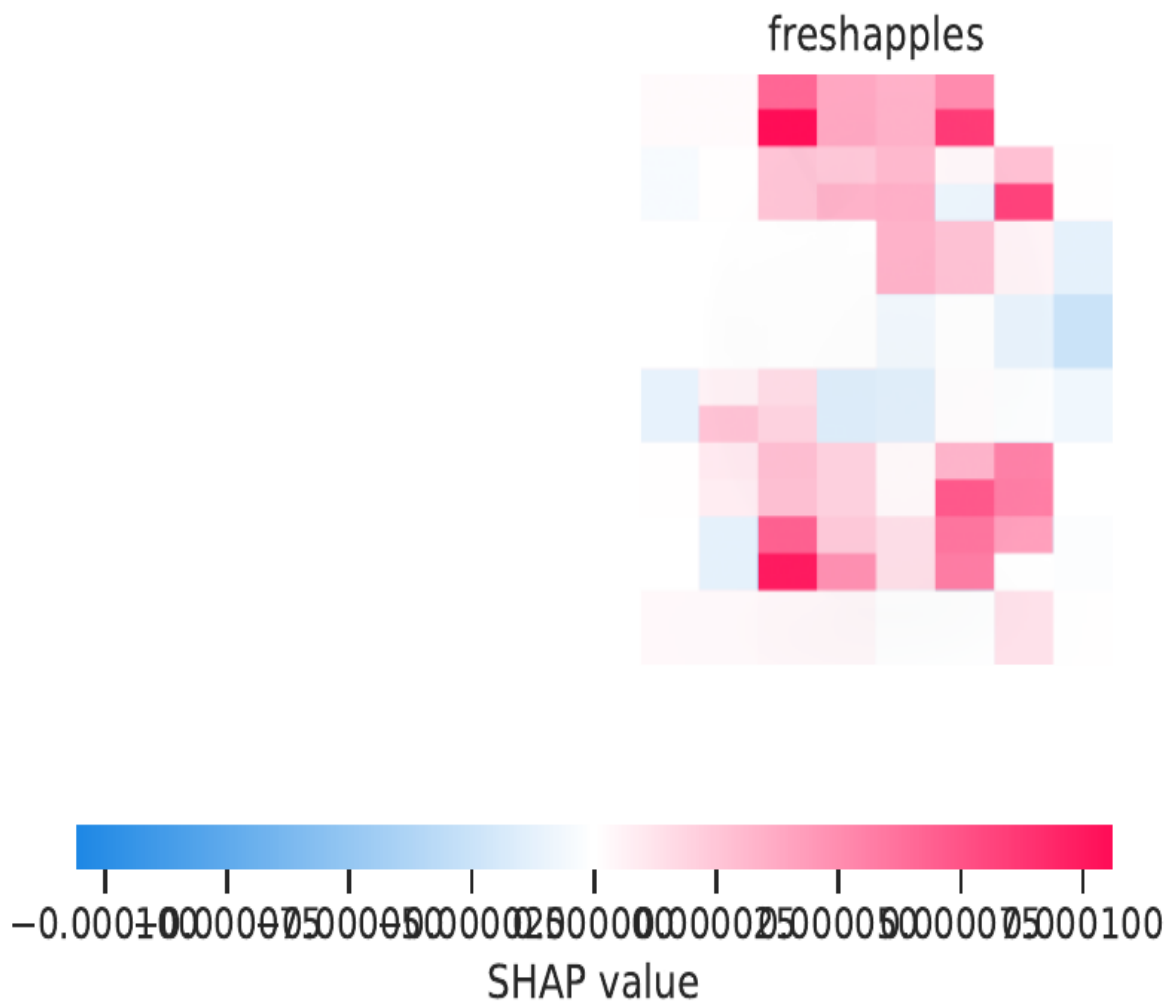


Figure 6: Grad-CAM Misclassification Examples

Figure 7: SHAP Explanations

SHAP image explanations showing pixel-level attributions for each model on the same example image. Pink and red pixels contributed positively to the predicted class; blue pixels worked against it. The clarity and spatial spread of these attributions differ between architectures, which is itself informative.

Custom CNN:**Figure 7: SHAP Explanation - Custom CNN***Figure 7a: SHAP Explanation — Custom CNN (freshapples class)*

MobileNetV2:**Figure 7: SHAP Explanation - MobileNetV2***Figure 7b: SHAP Explanation — MobileNetV2 (freshapples class)*

ResNet50z

Figure 7: SHAP Explanation - ResNet50

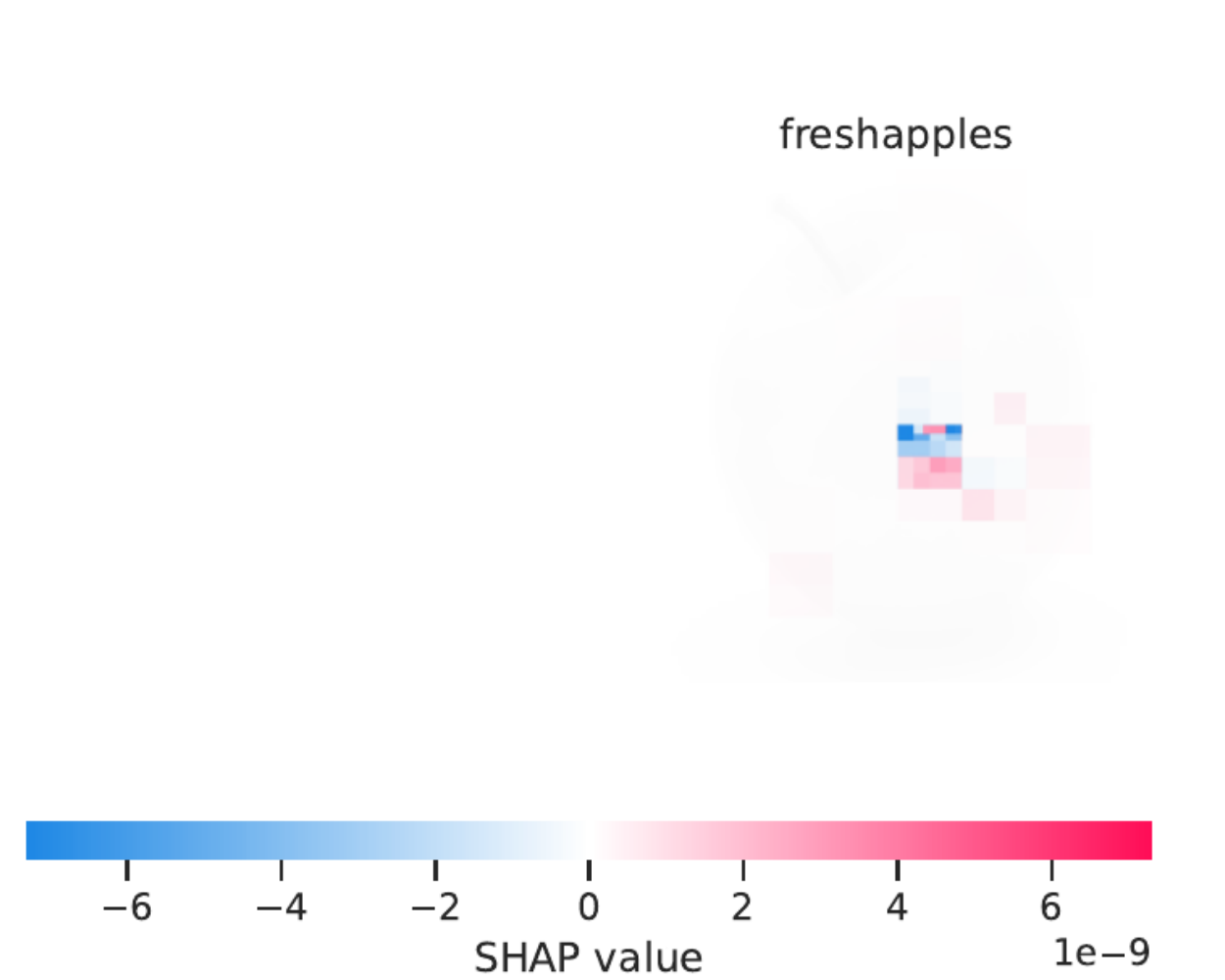


Figure 7c: SHAP Explanation — ResNet50 (freshapples class)

Table 1: Model Performance Comparison

A summary table bringing together accuracy, weighted F1-score, training duration, and parameter count for all three models — the main reference for comparing overall performance across architectures.

Table 2: Per-Class Classification Report

A detailed per-class breakdown of precision, recall, and F1-score across all 22 categories for each model. This is where fine-grained performance differences — which classes each model handles well, and which it struggles with — become visible.

13. Links

Github Link:

Kaggle Notebook Link: <https://www.kaggle.com/code/pankajkumawat23/cnn-final-file>

Dataset Link: <https://www.kaggle.com/datasets/swoyam2609/fresh-and-stale-classification>

Github Repository Link : <https://github.com/pankajkumawat23/Fresh-Fruit-Detection-Classification>